

ANSIBLE CHALLENGE

CI/CD PIPELINE PROJECT DOCUMENTATION

Terraform + Ansible + Jenkins (AWS)

1 Project Overview

This project automates infrastructure provisioning and configuration using a CI/CD pipeline.

- Terraform → creates AWS infrastructure
 - Ansible → configures servers
 - Jenkins → orchestrates the full pipeline
-

2 Files & Directories Created (IMPORTANT SECTION)

Terraform Files

File	Purpose
main.tf	Defines AWS EC2 instances (frontend & backend)
variables.tf	Stores reusable variables
output.tf	Outputs instance details
inventory.tpl	Template to generate Ansible inventory dynamically

Ansible Files

File	Purpose
inventory	Dynamic inventory generated by Terraform
site.yml	Main Ansible playbook
roles/common/tasks/main.yml	OS-level configurations
roles/frontend/tasks/main.yml	Nginx installation & config
roles/frontend/templates/nginx.conf.j2	Nginx reverse proxy template
roles/backend/tasks/main.yml	Netdata installation

Jenkins File

File	Purpose
Jenkinsfile	CI/CD pipeline definition

Project Directory Structure

ansible_project/

```

├── CI_pipeline/
│   ├── Terraform/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── output.tf
│   │   └── inventory.tpl
│   └──
└── Ansible/

```

```
|   |—— inventory
|   |—— site.yml
|   └── roles/
|       |—— common/
|       |   └── tasks/main.yml
|       |—— frontend/
|       |   └── tasks/main.yml
|       |   └── templates/nginx.conf.j2
|       └── backend/
|           └── tasks/main.yml
|
└── Jenkinsfile
```

Terraform Configuration Explained

◆ main.tf

- Creates 2 EC2 instances
 - Amazon Linux → frontend
 - Ubuntu → backend
- Assigns:
 - Security groups
 - Key pair
 - Tags & hostnames
- [Code:](#)

```
provider "aws" {  
    region = var.region  
}
```

```
resource "aws_instance" "frontend" {  
    ami          = "ami-0532be01f26a3de55" # Amazon Linux 2  
    instance_type = "t2.micro"  
    key_name     = var.key_name  
  
    tags = {  
        Name = "c8.local"  
    }  
}
```

```
resource "aws_instance" "backend" {  
    ami          = "ami-0b6c6ebed2801a5cb" # Ubuntu 21.04  
    instance_type = "t2.micro"  
    key_name     = var.key_name  
  
    tags = {  
        Name = "u21.local"  
    }  
}
```

```
Code Blame 23 lines (19 loc) · 456 Bytes

1  provider "aws" {
2    region = var.region
3  }
4
5  resource "aws_instance" "frontend" {
6    ami           = "ami-0532be81f26a3de55" # Amazon Linux 2
7    instance_type = "t2.micro"
8    key_name      = var.key_name
9
10   tags = {
11     Name = "c8.local"
12   }
13 }
14
15 resource "aws_instance" "backend" {
16   ami           = "ami-0b6c6e8d2801a5cb" # Ubuntu 21.04
17   instance_type = "t2.micro"
18   key_name      = var.key_name
19
20   tags = {
21     Name = "u21.local"
22   }
23 }
```

◆ variables.tf

- Stores:
 - AWS region
 - AMI IDs
 - Instance types
- Improves reusability and readability

Code:

```
variable "region" {
  default = "us-east-1"
}
```

```
variable "key_name" {
  default = "arj"
}
```

```
ansible_project / CI_pipeline / Terraform / variable.tf
arjmand9794 Add files via upload

Code Blame 7 lines (6 loc) · 94 Bytes

1 variable "region" {
2   default = "us-east-1"
3 }
4
5 variable "key_name" {
6   default = "arj"
7 }
```

◆ inventory.tpl

- Used to dynamically generate Ansible inventory
- Example output:

```
Code Blame 5 lines (4 loc) · 143 Bytes

1 [frontend]
2 c8.local ansible_host=${frontend_ip} ansible_user=ec2-user
3
4 [backend]
5 u21.local ansible_host=${backend_ip} ansible_user=ubuntu
```

[frontend]

c8.local ansible_host=<frontend-ip> ansible_user=ec2-user

[backend]

u21.local ansible_host=<backend-ip> ansible_user=ubuntu

✓ Eliminates manual inventory updates

```
anible_project / CI_pipeline / Ansible / roles / backend / tasks / main.yml
ajumand9794 Update Netdata installation and ensure service running
223db79

Code Blame 21 lines (19 loc) - 457 Bytes

1 - name: Install required packages (Ubuntu)
2 apt:
3   name:
4     - curl
5     - sudo
6   state: present
7   update_cache: yes
8   when: ansible_os_family == "Debian"
9
10 - name: Install Netdata (official installer)
11 shell: |
12   curl -fsSL https://get.netdata.cloud/kickstart.sh | bash
13 args:
14   creates: /usr/sbin/netdata
15   become: yes
16
17 - name: Ensure Netdata is running
18 service:
19   name: netdata
20   state: started
21   enabled: yes
```

5 Ansible Configuration Explained

◆ inventory

[all:vars]

ansible_ssh_common_args='-o StrictHostKeyChecking=no'

✓ Prevents SSH host key verification failure in Jenkins

```
anible_project / CI_pipeline / Ansible / inventory
ajumand9794 Update inventory
a43212b - 49 minutes ago History

Code Blame 6 lines (6 loc) - 244 Bytes

1 [all:vars]
2 ansible_ssh_common_args='-o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null'
3
4 [frontend]
5 c8.local ansible_host=100.27.217.231 ansible_user=ec2-user
6
7 [backend]
8 s21.local ansible_host=204.236.233.3 ansible_user=root
```

◆ site.yml (Main Playbook)

- hosts: all

become: yes

roles:

- common

- hosts: frontend

become: yes

roles:

- frontend

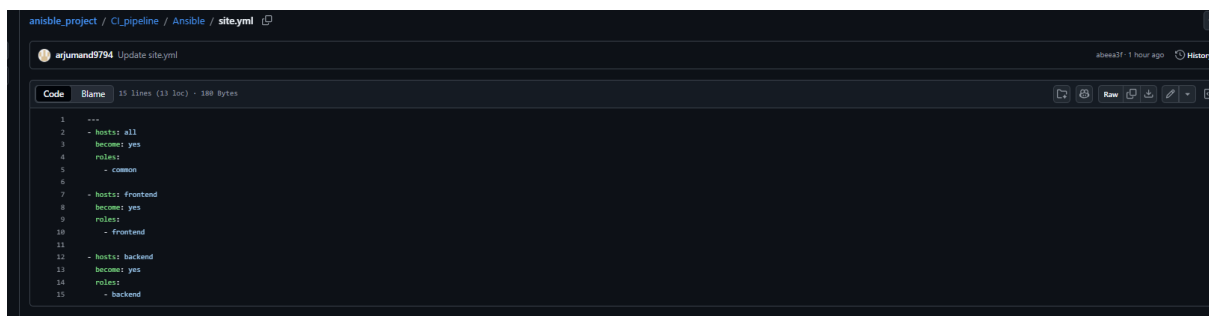
- hosts: backend

become: yes

roles:

- backend

- ✓ Executes roles in correct order
- ✓ Ensures OS configs before app setup



```
1 ---
2 - hosts: all
3   become: yes
4   roles:
5     - common
6
7 - hosts: frontend
8   become: yes
9   roles:
10     - frontend
11
12 - hosts: backend
13   become: yes
14   roles:
15     - backend
```

6 Ansible Roles Explained

◆ Common Role (roles/common)

Purpose: Base OS configuration

Tasks:

- Disable SELinux (Amazon Linux)
- Stop firewalld if present
- Stop UFW on Ubuntu

- ✓ Uses OS-specific conditions
 - ✓ Prevents service-related failures
-

◆ Frontend Role (roles/frontend)

Purpose: Configure Nginx as reverse proxy

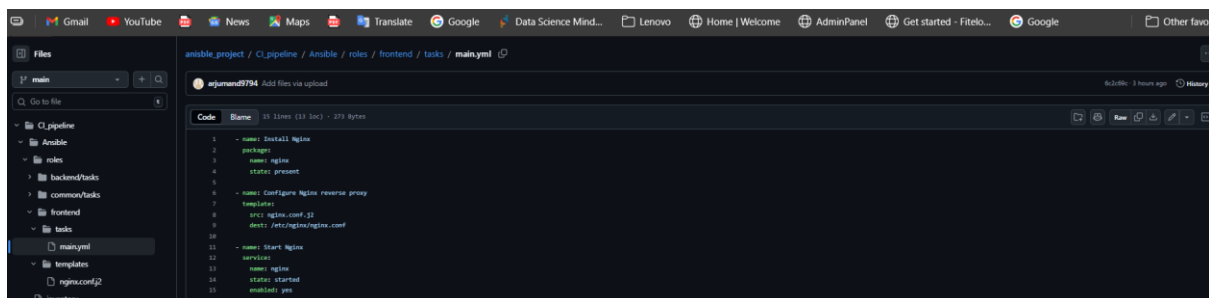
Tasks:

- Install Nginx
- Apply reverse proxy configuration
- Start & enable Nginx service

nginx.conf.j2

```
upstream backend_netdata {  
  
    server {{ hostvars[groups['backend']][0].ansible_host }}:19999;  
  
}
```

- ✓ Routes traffic from port 80 → 19999



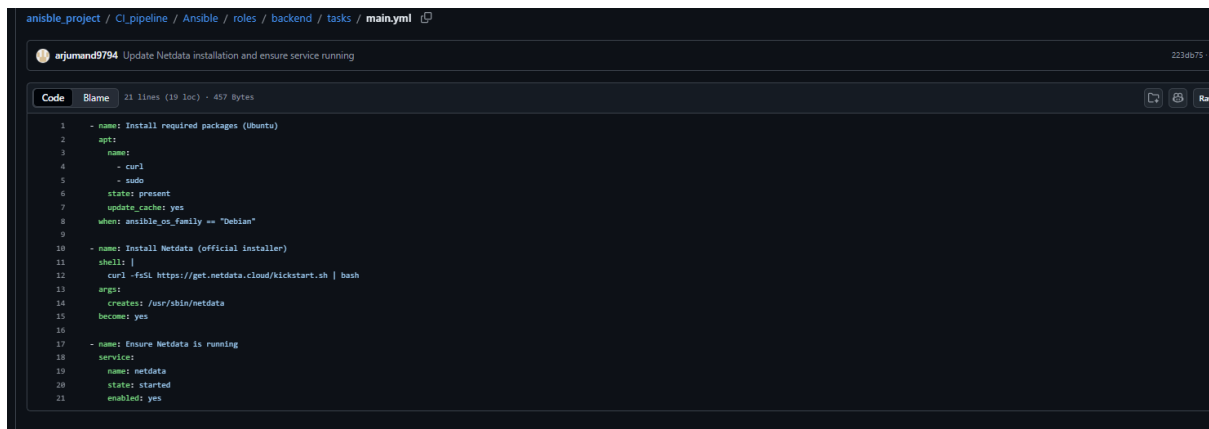
◆ Backend Role (roles/backend)

Purpose: Install Netdata monitoring

Tasks:

- Install Netdata using official script
- Ensure Netdata service is running

✓ Netdata runs on port 19999

A screenshot of a code editor showing an Ansible playbook snippet. The file path is 'ansible_project / CI_pipeline / Ansible / roles / backend / tasks / main.yml'. The snippet is titled 'Update Netdata installation and ensure service running'. It contains several tasks: 1. 'name: Install required packages (Ubuntu)' with 'apt' module and 'name: curl'. 2. 'name: state: present' with 'update_cache: yes' and 'when: ansible_os_family == "Debian"'. 3. 'name: Install Netdata (official installer)' with 'shell: | curl -fsSL https://get.netdata.cloud/kickstart.sh | bash'. 4. 'name: Ensure Netdata is running' with 'service: netdata', 'state: started', and 'enabled: yes'.

```
1 - name: Install required packages (Ubuntu)
2 apt:
3   name:
4     - curl
5   - sudo
6   state: present
7   update_cache: yes
8   when: ansible_os_family == "Debian"
9
10 - name: Install Netdata (official installer)
11 shell: |
12   curl -fsSL https://get.netdata.cloud/kickstart.sh | bash
13
14 args:
15   creates: /usr/sbin/netdata
16   become: yes
17
18 - name: Ensure Netdata is running
19 service:
20   name: netdata
21   state: started
22   enabled: yes
```

7 Jenkins Pipeline Explained

◆ Jenkinsfile Stages

Stage	Purpose
Checkout	Pull latest GitHub code
Terraform Init	Initialize Terraform
Terraform Apply	Provision infrastructure
Ansible Configure	Configure EC2 instances

◆ Jenkinsfile (Final Version)

pipeline {

agent any

environment {

AWS_DEFAULT_REGION = 'us-east-1'

}

```
stages {
```

```
  stage('Checkout') {
```

```
    steps {
```

```
      git branch: 'main',
```

```
      url: 'https://github.com/arjumand9794/anisble_project.git'
```

```
    }
```

```
  }
```

```
  stage('Terraform Init') {
```

```
    steps {
```

```
      withCredentials([
```

```
        string(credentialsId: 'access_key', variable: 'AWS_ACCESS_KEY_ID'),
```

```
        string(credentialsId: 'secrete_key', variable: 'AWS_SECRET_ACCESS_KEY')
```

```
      ]) {
```

```
        dir('CI_pipeline/Terraform') {
```

```
          sh 'terraform init'
```

```
        }
```

```
      }
```

```
    }
```

```
  }
```

```
  stage('Terraform Apply') {
```

```
steps {
  withCredentials([
    string(credentialsId: 'access_key', variable: 'AWS_ACCESS_KEY_ID'),
    string(credentialsId: 'secrete_key', variable:
'AWS_SECRET_ACCESS_KEY')
  ]) {
    dir('CI_pipeline/Terraform') {
      sh 'terraform apply -auto-approve'
    }
  }
}
```

```
stage('Ansible Configure') {
  steps {
    withCredentials([
      sshUserPrivateKey(
        credentialsId: 'arj.pem',
        keyFileVariable: 'SSH_KEY',
        usernameVariable: 'SSH_USER'
      )
    ]) {
      dir('CI_pipeline/Ansible') {
        sh '''
```

```
chmod 600 $SSH_KEY
```

```
ansible-playbook -i inventory site.yml \
```

```
--private-key $SSH_KEY
```

```
'''
```

```
}
```

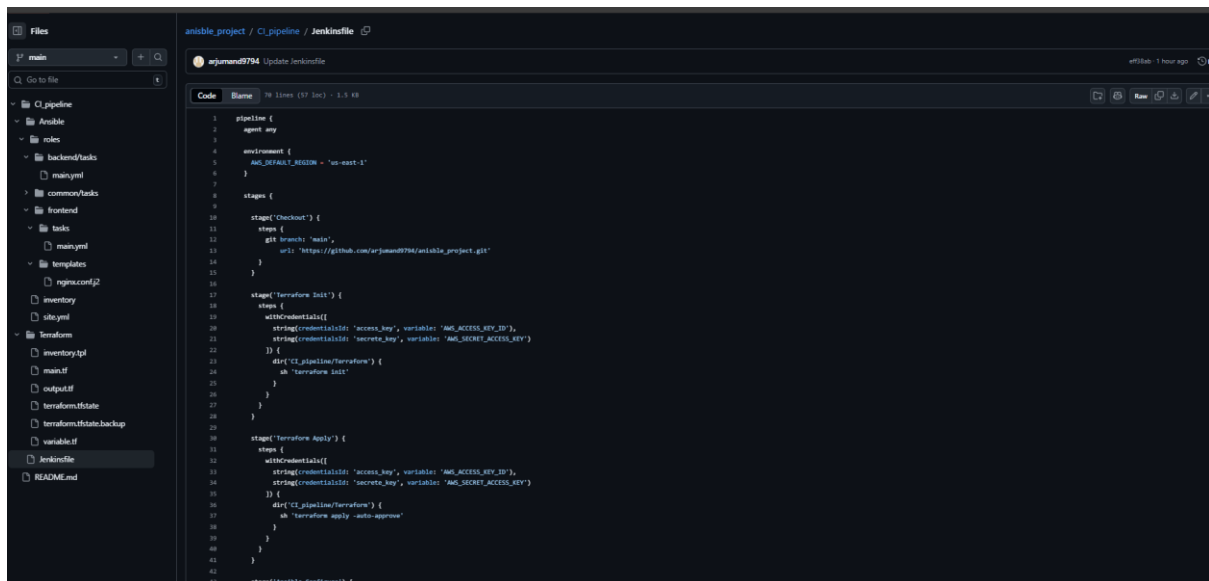
```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

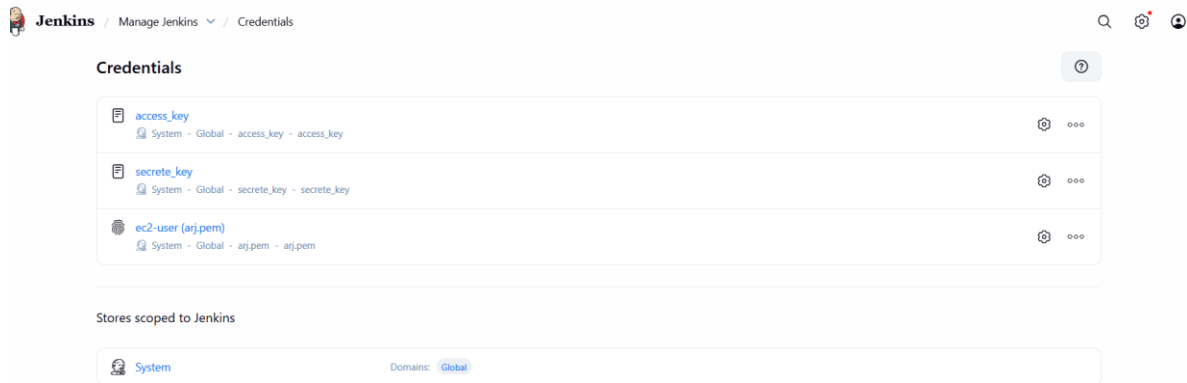


The screenshot shows a Jenkinsfile configuration for a CI/CD pipeline. The pipeline is named 'pipeline' and has a single agent 'any'. The environment is set to 'Ansible' with 'AMC_DEFAULT_REGION' set to 'us-east-1'. The pipeline consists of three stages: 'Checkout', 'Terraform Init', and 'Terraform Apply'. The 'Checkout' stage checks out the code from a GitHub repository. The 'Terraform Init' stage initializes Terraform with credentials for 'secret_key' and 'secret_access_key'. The 'Terraform Apply' stage applies the Terraform configuration with the same credentials. The pipeline is configured to run on a Jenkins agent named 'any'.

```
1 pipeline {
2   agent any
3
4   environment {
5     AMC_DEFAULT_REGION = "us-east-1"
6   }
7
8   stages {
9
10    stage('Checkout') {
11      steps {
12        git branch: 'main',
13          url: 'https://github.com/arjmand9794/ansible_project.git'
14      }
15    }
16
17    stage('Terraform Init') {
18      steps {
19        withCredentials([
20          string(credentials: 'secret_key', variable: 'AMC_ACCESS_KEY_ID'),
21          string(credentials: 'secret_key', variable: 'AMC_SECRET_ACCESS_KEY')
22        ]) {
23          sh('terraform init')
24        }
25      }
26    }
27
28    stage('Terraform Apply') {
29      steps {
30        withCredentials([
31          string(credentials: 'secret_key', variable: 'AMC_ACCESS_KEY_ID'),
32          string(credentials: 'secret_key', variable: 'AMC_SECRET_ACCESS_KEY')
33        ]) {
34          sh('terraform apply -auto-approve')
35        }
36      }
37    }
38
39    stage('Ansible Configure') {
40
41    }
42  }
43 }
```

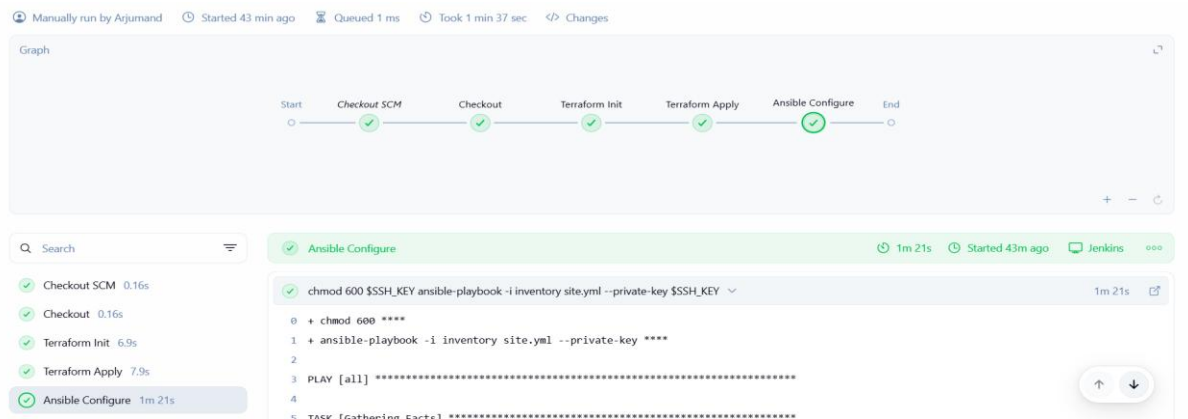
→Add credentials ssh_key type of credentials in global and add pem key and give as ec2-user

→Add secrete key and access separately with secrete text as type in global credentials

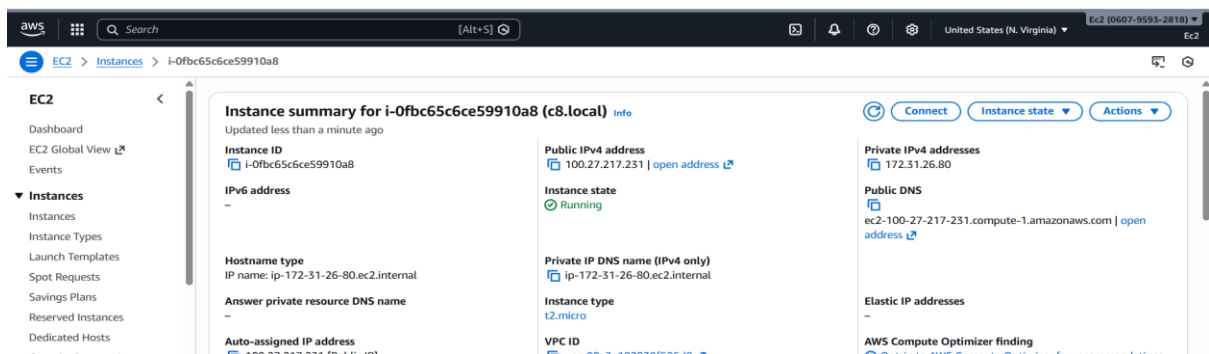


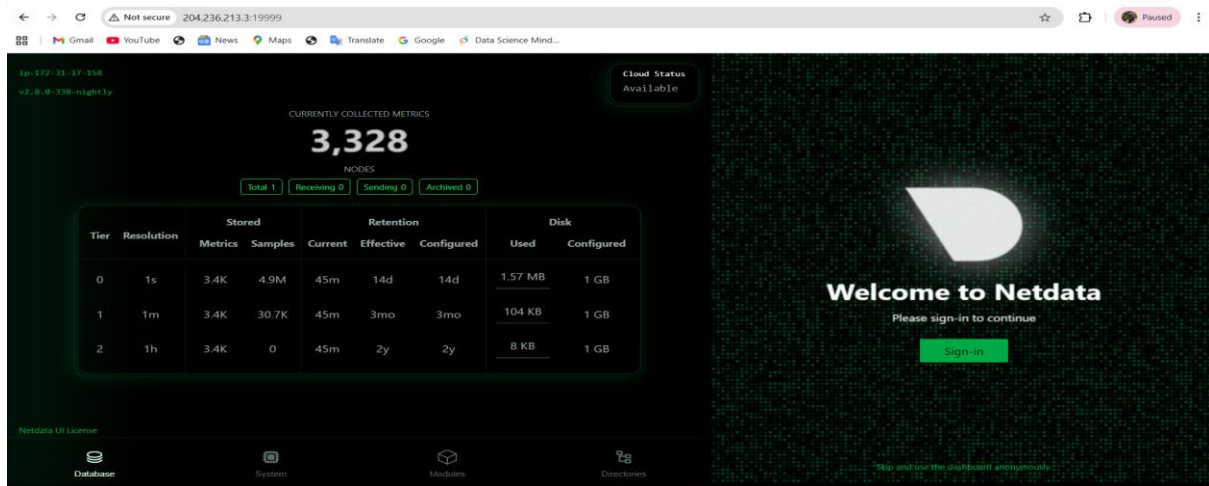
8 Final Result

- Terraform creates infrastructure
- Inventory generated automatically
- Ansible configures:
 - Frontend → Nginx reverse proxy
 - Backend → Netdata monitoring
- Jenkins pipeline completes successfully

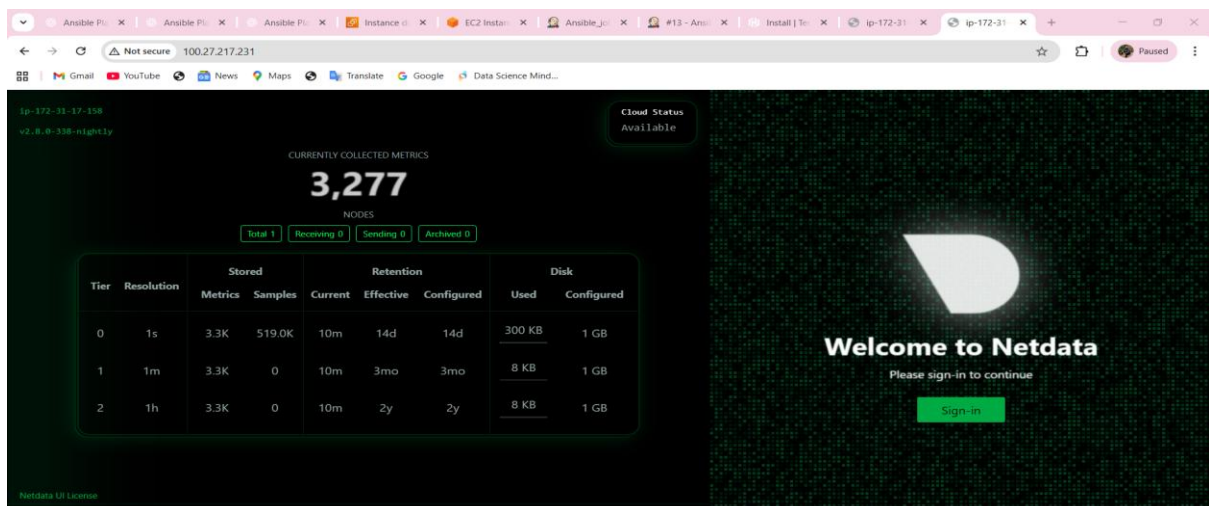
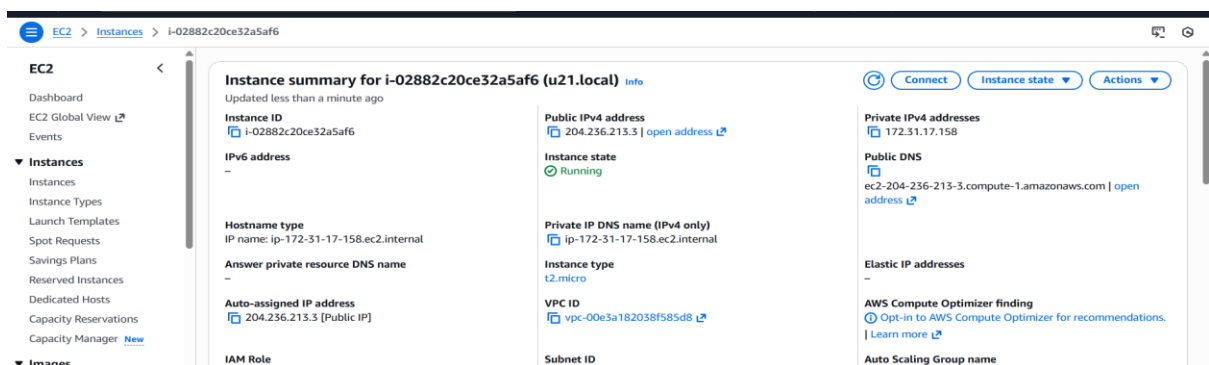


→ c8.local with public Id routed to 19999





→ u21.local with public ip routed to 19999 port number



9 Key Skills Demonstrated

- ✓ Infrastructure as Code
- ✓ CI/CD pipeline design
- ✓ Configuration management
- ✓ AWS automation
- ✓ Real-world troubleshooting